GODOT 4.X • GAME 08

Game 8 – Chunk Style শেখার গাইড

সহজ বাংলায়, ধীরে ধাপে, আসল কোড স্ক্যান করে বানানো একটি লম্বা PDF গাইড

উদ্দেশ্য

তুমি কী বানিয়েছ, কেন এই order-এ বানিয়েছ, আর কীভাবে chunk-by-chunk চিন্তা করলে system clean থাকে — সেটা এক জায়গায় ধরে রাখা।

ভিত্তি

এই গাইড তোমার আপলোড করা `player.gd`, `enemy.gd`, `bullet.gd`, `pool_manager.gd` ইত্যাদি স্ক্রিপ্ট স্ক্যান করে লেখা।

শৈলী

এখানে ইচ্ছা করে "একসাথে সবকিছু" শেখানো হয়নি। বরং একজন ভালো developer যেভাবে scope control করে এগোয়, সেভাবেই explanation সাজানো হয়েছে।

নিয়ম

Structure আগে, cleverness পরে। Stable chunk আগে, polish পরে। একসাথে বেশি feature যোগ করলে শেখা ভেঙে যায়।

এই ডকুমেন্টের goal হলো তোমাকে code মুখস্থ করানো না; বরং code-এর পেছনের mental model clear করা।

সূচিপত্র

1. এই PDF কীভাবে ব্যবহার করবে
2. গেম ৮-এর final scope
3. Chunk progress map
4. High-level scene architecture
5. Phase-by-phase build explanation
6. Pooling mini-training
7. AAA-style thinking within scope
8. Uploaded code scan: কী ভালো, কী risky
9. Cleaned final code appendix

১. এই PDF কীভাবে ব্যবহার করবে

> সেরা ব্যবহার-পদ্ধতি: পুরো PDF এক বসায় শেষ করার জন্য না। বরং phase ধরে ধরে পড়বে। প্রতিটি phase-এর পরে Godot project খুলে মিলিয়ে দেখবে।

- প্রথমে **Chunk Progress Map** দেখো — এতে পুরো game-এর skeleton বুঝবে।
- তারপর Phase 1 → Phase 6 পর্যন্ত পড়ে দেখো কোন phase কোন problem solve করছে।
- শেষে appendix-এর full code দেখো। Full code আগে দেখলে অনেক সময় structure চোখে পড়ে না।
- যে phase-এ তোমার confusion বেশি ছিল, সেই section-এ ফিরে গিয়ে "ভুলগুলো" বক্সগুলো পড়ো।

> সবচেয়ে গুরুত্বপূর্ণ কথা: একজন beginner যখন বড় system বানাতে যায়, সে সাধারণত code-এর line দেখে। একজন strong developer প্রথমে *responsibility* দেখে — "এই কাজটা player করবে, না enemy করবে, না game manager করবে?"

২. Game 8-এর final scope

এই game-টা full AAA game না। কিন্তু এর ভিতরে এমন কিছু system আছে যেগুলোকে clean foundation দিয়ে বানালে পরে বড় game-এর দিকে উঠা সহজ হয়। তোমার final built scope roughly এইরকম:

- Top-down player movement
- Shooting
- Bullet movement + bullet collision
- Enemy chase + attack
- Enemy finite state machine-এর simple version (CHASE ↔ ATTACK)
- Enemy spawner
- Score
- UI: health, score, game over
- Bullet pooling mini-upgrade

খেয়াল করো: এখানে pathfinding নেই, navmesh নেই, obstacle avoidance নেই, animation state নেই, save system নেই। এগুলো না থাকলেও এই game-এ শেখার value অনেক, কারণ তুমি **system layering** শিখেছ।

৩. Chunk Progress Map

Chunk	What was built	Status
১A	Player Movement	সম্পন্ন
২A	Facing / Aim Direction Base	সম্পন্ন

Chunk	What was built	Status
৩**A**	Shooting Input Trigger	সম্পন্ন
৩**B**	Bullet Spawn	সম্পন্ন
৩**C**	Bullet Movement	সম্পন্ন
৩**D**	Bullet Collision	সম্পন্ন
৪**A**	Enemy Base	সম্পন্ন
৪**B**	Enemy Health & Damage	সম্পন্ন
৪**C**	Enemy Movement / Chase	সম্পন্ন
৪**D**	Enemy Attack	সম্পন্ন
৪**E**	Enemy FSM	সম্পন্ন
৫**A**	Enemy Spawner	সম্পন্ন
৫**B**	Score System	সম্পন্ন
৫**C**	GameManager signal cleanup	সম্পন্ন
৬**A**	Health UI	সম্পন্ন
৬**B**	Score UI	সম্পন্ন
৬**C**	Game Over + Restart	সম্পন্ন
P1	Bullet Pooling mini-training	সম্পন্ন

> এই **map-**এর **value** কী? তুমি যখন game build করছিলে, numbering যেমন 4E, 5A — এগুলো psychological support দিয়েছে। বড় project-এ এই numbering trivial না; এটা scope control করে।

৪. High-Level Scene Architecture

তোমার game-এর final idea roughly এই structure-এর মধ্যে বসে:

```
Main (Node2D)
├── Player
├── PoolManager
├── BulletContainer
├── EnemySpawner
├── GameManager
├── UI
└── Timer / other helper nodes
```

Player-এর responsibility

- Input পড়া
- Move করা
- Shoot request করা
- নিজের health কমানো

Enemy-এর responsibility

- Player খুঁজে follow করা
- Attack range check করা
- Damage দেওয়া
- নিজের health manage করা

GameManager-এর responsibility

- Score রাখা
- Signal দিয়ে UI-কে জানানো

PoolManager-এর responsibility

- Bullet create করা একবার
- পরে inactive bullet ফেরত দেওয়া

AAA-style মূল নিয়ম: "যে node-এর কাজ না, সেই node সেখানে logic রাখবে না।" এই rule যত তাড়াতাড়ি ধরতে পারবে, তত কম spaghetti code হবে।

Phase 1 – Player Movement

Goal: character smoothly move করবে। এখানে shooting, enemy, UI — কিছুই দরকার ছিল না। শুধু movement stable হওয়া দরকার ছিল।

এই phase-এ মূল mental model ছিল:

- InputMap থেকে input strength নেওয়া
- X ও Y দিয়ে direction বানানো
- Direction normalize করা যাতে diagonal speed বেশি না হয়
- Velocity = direction * speed
- `move_and_slide()` দিয়ে movement চালানো

```
extends CharacterBody2D

@onready var gun_pivot := $Marker2D
@onready var shoot_timer := $ShootTimer

const SPEED := 300.0

var gun_facing_dir: Vector2 = Vector2.RIGHT
var player_life := 30
var pool_manager
```

```
func _ready() -> void:
	add_to_group("player")

func setup(_pool) -> void:
	pool_manager = _pool

func _physics_process(_delta: float) -> void:
	var player_input_dir := _get_dir()
	velocity = player_input_dir * SPEED

	if player_input_dir != Vector2.ZERO:
		gun_facing_dir = player_input_dir

	if Input.is_action_pressed("shoot") and shoot_timer.is_stopped():
		_shoot()

	move_and_slide()
	_update_gun_pivot()

func _get_dir() -> Vector2:
	var direction := Vector2(
		Input.get_action_strength("move_right") -
Input.get_action_strength("move_left"),
		Input.get_action_strength("move_down") -
Input.get_action_strength("move_up")
	)
	return direction.normalized()
```

সহজ ভাষায়: direction বলে “কোন দিকে যাব”, speed বলে “কত জোরে যাব”। এই দুইটা মিলে velocity হয়।

Common beginner mistake: diagonal input-এ normalize না করলে player সোজা চলার চেয়ে কোণাকুণি গেলে দ্রুত চলে।

Phase 2 – Facing Direction / Gun Pivot

Goal: player যেদিকে শেষবার move করেছে, gun pivot সেইদিকে ঘুরে থাকবে। এই phase shooting-এর preparation।

এখানে খুব গুরুত্বপূর্ণ concept ছিল — **movement direction** আর **last facing direction** এক জিনিস না। Player থেমে গেলে movement zero হয়, কিন্তু gun direction zero হয়ে গেলে shoot broken হয়ে যায়। তাই last non-zero direction আলাদা করে রাখা হয়।

```
var gun_facing_dir : Vector2 = Vector2.RIGHT

if player_input_dir != Vector2.ZERO:
	gun_facing_dir = player_input_dir

func _update_gun_pivot():
```

```
        gun_pivot.position = gun_facing_dir * 90.0
        gun_pivot.rotation = gun_facing_dir.angle()
```

> কেন **useful?** Player থেমে গেলেও তুমি শেষবার যেদিকে move করেছিলে, সেদিকেই shoot করতে পারো।

Phase 3 – Shooting System (Trigger → Spawn → Move → Hit)

এই phase-টা তুমি ছোট ছোট ভাগে করেছিলে, আর সেটাই ছিল সঠিক সিদ্ধান্ত। যদি একসাথে bullet spawn + movement + collision + pooling করতে যেতে, তাহলে confusion বাড়ত।

৩A – Trigger

প্রথমে শুধুই button press বা mouse press detect করা হয়েছে। Timer দিয়ে fire rate control করা হয়েছে।

৩B – Spawn

তারপর bullet scene instantiate করে spawn position ঠিক করা হয়েছে। এখানেই spawn offset বুঝতে গিয়ে তোমার বেশ confusion হয়েছিল।

৩C – Move

তারপর bullet শুধু forward move করেছে।

৩D – Collision

সবশেষে bullet enemy-কে hit করলে damage গেছে।

```
extends Area2D

@export var speed := 600.0
@export var lifetime := 1.5
@export var damage := 10

var direction := Vector2.ZERO
var active := false
var life_timer := 0.0

func activate(pos: Vector2, dir: Vector2) -> void:
        global_position = pos
        direction = dir.normalized()
        life_timer = 0.0
        active = true
        visible = true
        set_deferred("monitoring", true)
        set_physics_process(true)

func _physics_process(delta: float) -> void:
```

```
        if not active:
                return

        position += direction * speed * delta
        life_timer += delta

        if life_timer >= lifetime:
                deactivate()

func _on_body_entered(body: Node) -> void:
        if not active:
                return

        if body.is_in_group("enemy") and body.has_method("take_damage"):
                body.take_damage(damage)

        deactivate()

func deactivate() -> void:
        active = false
        visible = false
        set_deferred("monitoring", false)
        set_physics_process(false)
```

খুব গুরুত্বপূর্ণ **lesson:** spawn position visually ঠিক দেখালেই logic ঠিক — এমন না। Node origin, sprite center, spawn offset — সব একসাথে বুঝতে হয়।

তোমার **uploaded snapshot-**এ একটি **inconsistency:** bullet script `take_damage` method খুঁজছে, কিন্তু uploaded enemy script-এ method ছিল `_take_damage`। এর মানে এই snapshot 그대로 চালালে bullet damage কাজ নাও করতে পারে।

Phase 4 – Enemy System (Base → Health → Chase → Attack → FSM)

Game 8-এর আসল weight ছিল enemy system-এ। কারণ enemy মানে শুধু move করা না; condition বুঝে behavior বদলানো।

৪A – Enemy Base

Enemy scene আলাদা করা হয়েছে। Group-এ যোগ করা হয়েছে।

৪B – Health & Damage

Bullet hit করলে enemy health কমেছে। Health zero হলে enemy মরে গেছে।

৪C – Chase

Enemy player-এর দিকে এগিয়েছে। এখানে vector math ছিল key concept:

```
var direction = (player.global_position - global_position).normalized()
velocity = direction * speed
```

এই **line-**এর মানে: “Player-এর position আর enemy-এর position-এর difference নাও। তারপর direction unit vector বানাও। তারপর speed দিয়ে multiply করো। ”

৪D – Attack

Distance কম হলে enemy attack করেছে। Attack timer দিয়ে damage spam আটকানো হয়েছে।

৪E – FSM

এখানে same behavior-কে organized করা হয়েছে। আগে chase + attack logic একসাথে মিশে ছিল। FSM আনার পরে enemy-এর brain readable হয়েছে।

```
extends CharacterBody2D

@export var max_health := 30
@export var speed := 80
@export var damage := 10
@export var attack_range := 140.0

var can_attack := true
var health: int
var player

enum State {
	CHASE,
	ATTACK
}

var state: State = State.CHASE

@onready var attack_timer := $AttackTimer

func _ready() -> void:
	add_to_group("enemy")
	health = max_health
	player = get_tree().get_first_node_in_group("player")

func _physics_process(_delta: float) -> void:
	if player == null:
		return

	var distance = global_position.distance_to(player.global_position)

	match state:
		State.CHASE:
			update_chase(distance)
```

```
                State.ATTACK:
                        update_attack(distance)

        move_and_slide()

func update_chase(distance: float) -> void:
        var direction = (player.global_position - global_position).normalized()
        velocity = direction * speed

        if distance < attack_range:
                state = State.ATTACK

func update_attack(distance: float) -> void:
        velocity = Vector2.ZERO

        if can_attack:
                _attack()

        if distance > attack_range:
                state = State.CHASE

func _attack() -> void:
        if player.has_method("take_damage"):
                player.take_damage(damage)
                can_attack = false
                attack_timer.start()

func _on_attack_timer_timeout() -> void:
        can_attack = true

func take_damage(amount: int) -> void:
        health -= amount
        print("[Enemy] enemy HP: ", health)

        if health <= 0:
                _die()

func _die() -> void:
        var gm = get_tree().get_first_node_in_group("game_manager")
        if gm:
                gm.add_score(10)

        print("[Enemy] enemy died")
        queue_free()
```

FSM beginner-friendly truth: FSM নতুন behavior বানায় না। আগে যা কাজ করছিল, সেটাকে সুন্দর করে section করে।

Common mistake: একসাথে PATROL, CHASE, ATTACK, SEARCH সব state ঢুকিয়ে দেওয়া। তোমার জন্য CHASE ↔ ATTACK first step হিসেবে ঠিক ছিল।

Phase 5 – Game Systems (Spawner, Score, GameManager)

একটা game “system” হয় তখন, যখন scene-এ manually enemy রেখে খেলতে হয় না। Timer থেকে spawn হয়, enemy মরে score বাড়ে, UI সেটা দেখায়।

৫A – Enemy Spawner

Timer timeout-এ enemy instantiate করা হয়েছে। Random position থেকে spawn হয়েছে।

```
extends Node2D

@export var enemy_scene: PackedScene

func _on_timer_timeout() -> void:
	var enemy = enemy_scene.instantiate()

	enemy.global_position = Vector2(
		randf_range(100, 600),
		randf_range(100, 500)
	)

	get_tree().current_scene.add_child(enemy)
```

৫B – Score System

Enemy die করলে GameManager score বাড়িয়েছে।

```
extends Node

signal score_changed(new_score)

var score := 0

func _ready() -> void:
	add_to_group("game_manager")

func add_score(amount: int) -> void:
	score += amount
	print("score: ", score)
	score_changed.emit(score)
```

৫C – Why GameManager mattered

যদি enemy নিজে score label update করত, তাহলে architecture dirty হতো। Enemy-এর কাজ শুধু “আমি মরেছি, score বাড়াও” বলা। Score কোথায় দেখানো হবে সেটা GameManager/UI-এর কাজ।

> **AAA-style thought:** যতদিন scope ছোট, group lookup acceptable। Project বড় হলে dependency injection বা explicit references better।

Phase 6 – UI (Health, Score, Game Over)

এই phase-এ gameplay visible হয়েছে। Backend আগে থেকেই ছিল, এখন player health আর score চোখে দেখা যাচ্ছে।

```
extends Control

@onready var health_label = $HealthLabel
@onready var score_label = $ScoreLabel
@onready var game_over_label = $GameOverLabel

var player
var gm

func _ready() -> void:
	player = get_tree().get_first_node_in_group("player")
	gm = get_tree().get_first_node_in_group("game_manager")
	_update_score(0)

	if gm:
		gm.score_changed.connect(_update_score)

	game_over_label.visible = false

func _process(_delta: float) -> void:
	if player:
		health_label.text = "Health: " + str(player.player_life)

	if player and player.player_life <= 0:
		game_over_label.visible = true

	if Input.is_action_just_pressed("restart"):
		get_tree().paused = false
		get_tree().reload_current_scene()

func _update_score(new_score: int) -> void:
	score_label.text = "Score: " + str(new_score)
```

UI script-এ তুমি দুই রকম pattern ব্যবহার করেছ:

- **Polling:** health প্রতি frame update হচ্ছে
- **Signal:** score only change হলে update হচ্ছে

> এখানে একটি **practical note:** ছোট project-এ health polling okay। বড় project-এ health change signal cleaner হয়।

Phase 7 – Game Over + Restart

Player life zero হলে game paused হয়েছে। UI game over label visible হয়েছে। Restart input scene reload করেছে।

এটা ছোট feature মনে হলেও, আসলে এখানেই full gameplay loop complete হয়:

- play
- damage
- death
- game over feedback
- restart

> এই **point-**এর পরে তুমি **technically** **“একটা ছোট complete game”** বানিয়ে ফেলেছ।

```
func _die():
        print("game over!")
        get_tree().paused = true

# UI side
if Input.is_action_just_pressed("restart"):
        get_tree().paused = false
        get_tree().reload_current_scene()
```

এর মানে তোমার game শুধু system demo না; এতে start → play → lose → restart loop আছে।

৭. Pooling Mini-Training

তুমি Game 8 শেষ করার পরে আরেকটা গুরুত্বপূর্ণ mini-session করেছিলে: **bullet pooling**। এটা খুব valuable ছিল, কারণ এখানে প্রথমবার তুমি object lifecycle control হাতে নিয়েছ।

Pooling-এর easy definition:

- আগে: shoot → bullet create → use → delete
- পরে: bullet আগে থেকে create → দরকারে activate → use → deactivate → আবার reuse

কেন **pooling** দরকার?

- প্রতি শটে নতুন object বানালে performance hit হতে পারে
- Reuse করলে memory churn কমে
- Game feel stable হয়

তুমি এখানে কী শিখেছ?

- `queue_free()` সরিয়ে `deactivate()` ব্যবহার
- `visible`, `monitoring`, `physics_process` manually control করা
- Pool full হয়ে গেলে null return করার simple version

```
extends Node

@export var bullet_scene: PackedScene
@export var pool_size := 30

var pool: Array = []
var container: Node

func setup(_container) -> void:
	container = _container
	create_pool()

func create_pool() -> void:
	for i in range(pool_size):
		var bullet = bullet_scene.instantiate()
		bullet.visible = false
		bullet.set_physics_process(false)
		container.add_child(bullet)
		pool.append(bullet)

func get_bullet():
	for bullet in pool:
		if not bullet.active:
			return bullet
```

```
        return null
```

```
extends Area2D

@export var speed := 600.0
@export var lifetime := 1.5
@export var damage := 10

var direction := Vector2.ZERO
var active := false
var life_timer := 0.0

func activate(pos: Vector2, dir: Vector2) -> void:
        global_position = pos
        direction = dir.normalized()
        life_timer = 0.0
        active = true
        visible = true
        set_deferred("monitoring", true)
        set_physics_process(true)

func _physics_process(delta: float) -> void:
        if not active:
                return

        position += direction * speed * delta
        life_timer += delta

        if life_timer >= lifetime:
                deactivate()

func _on_body_entered(body: Node) -> void:
        if not active:
                return

        if body.is_in_group("enemy") and body.has_method("take_damage"):
                body.take_damage(damage)

        deactivate()

func deactivate() -> void:
        active = false
        visible = false
        set_deferred("monitoring", false)
        set_physics_process(false)
```

Pooling-এর আসল **pain point:** object delete না করলে engine state নিজে reset হয় না। তাই তোমাকেই manually reset করতে হয়:

- visible
- active flag
- collision monitoring

- life timer
- direction

তুমি এখানে যে **version-**এ থেমেছ, সেটা ঠিক **decision** ছিল। Auto-expand pool আরও scalable, কিন্তু learning-stage-এ base pooling concept clear হওয়াই main win।

৮. একজন "AAA-minded" developer এই game-এ কীভাবে এগোয়

এই section-এর উদ্দেশ্য তোমাকে বোঝানো: AAA মানে সব feature ঢুকিয়ে দেওয়া না। AAA মানে **scope-**এর ভেতরের জিনিসটাকে **clean** করা।

ভুল **approach**	ভালো **approach**
একসাথে movement + shooting + enemy + UI + pooling	Phase-by-phase build, প্রতিটি phase test করা
Code paste করে পরে bug hunt	প্রতিটি নতুন system-এর পরে verification
সব responsibility player script-এ ঢোকানো	Player, Enemy, UI, Manager আলাদা করা
Feature count দিয়ে "advanced" ভাবা	Existing feature-এর code quality improve করা
Bug দেখলে random fix	Reproduce → isolate → fix → retest

সবচেয়ে বড় **lesson:** তুমি যখন restart করে phase-wise build করা শুরু করলে, তখনই learning curve clean হয়ে গেছে। এর মানে তোমার biggest growth এসেছে code complexity থেকে না, *build discipline* থেকে।

৯. Uploaded Code Scan — কী ভালো, কী risky

আমি তোমার uploaded snapshot দেখে কয়েকটা strong point আর কয়েকটা risky point পেয়েছি। এগুলো জানা দরকার, কারণ future project-এ এগুলো আবার সামনে আসবে।

যেগুলো ভালো

- Player, Enemy, Bullet, PoolManager, UI, GameManager আলাদা script-এ আছে
- Group usage আছে — `player`, `enemy`, `game_manager`
- Enemy-তে attack cooldown timer আছে
- GameManager signal দিয়ে score update করছে
- PoolManager basic pooling বুঝিয়েছে

যেগুলো **risky** বা **inconsistent**

- **Method name mismatch:** bullet script `take_damage` খুঁজে, enemy script-এ ছিল `_take_damage`

- **Unused export:** player script-এ `bullet_scene` আছে, কিন্তু pooling version-এ সেটা আর লাগছে না
- **Spawner ownership:** enemy spawner directly current scene-এ add করছে; ছোট game-এ okay, কিন্তু dedicated enemy container cleaner
- **Pool full** হলে **null:** learning-এর জন্য ঠিক, কিন্তু intense combat game-এ shot fail হতে পারে
- **UI polling:** health label প্রতি frame update হচ্ছে; small project okay, larger project-এ signal better

> **Important:** এগুলো "তুমি খারাপ কোড লিখেছ" টাইপ judgement না। এগুলো হচ্ছে natural next-step cleanup points। Beginner থেকে strong dev হওয়া মানে exactly এই inconsistencies নিজের চোখে ধরতে শেখা।

১০. Cleaned Final Code Appendix

নিচের code-গুলো তোমার uploaded scripts-এর educational cleanup version। মানে, same structure রাখা হয়েছে, কিন্তু naming inconsistency আর কয়েকটা readability issue fix করা হয়েছে যাতে future-এ বুঝতে সুবিধা হয়।

main.gd

```
extends Node2D

@onready var pool = $PoolManager
@onready var bullet_container = $BulletContainer
@onready var player = $Player

func _ready() -> void:
	pool.setup(bullet_container)
	player.setup(pool)
```

player.gd

```
extends CharacterBody2D

@onready var gun_pivot := $Marker2D
@onready var shoot_timer := $ShootTimer

const SPEED := 300.0

var gun_facing_dir: Vector2 = Vector2.RIGHT
var player_life := 30
var pool_manager

func _ready() -> void:
	add_to_group("player")

func setup(_pool) -> void:
	pool_manager = _pool

func _physics_process(_delta: float) -> void:
	var player_input_dir := _get_dir()
	velocity = player_input_dir * SPEED

	if player_input_dir != Vector2.ZERO:
		gun_facing_dir = player_input_dir

	if Input.is_action_pressed("shoot") and shoot_timer.is_stopped():
		_shoot()

	move_and_slide()
	_update_gun_pivot()

func _get_dir() -> Vector2:
	var direction := Vector2(
```

```
                Input.get_action_strength("move_right") -
Input.get_action_strength("move_left"),
                Input.get_action_strength("move_down") -
Input.get_action_strength("move_up")
        )
        return direction.normalized()

func _update_gun_pivot() -> void:
        gun_pivot.position = gun_facing_dir * 90.0
        gun_pivot.rotation = gun_facing_dir.angle()

func _shoot() -> void:
        if pool_manager == null:
                return

        var bullet = pool_manager.get_bullet()
        if bullet == null:
                return

        var direction := gun_facing_dir.normalized()
        if direction == Vector2.ZERO:
                direction = Vector2.RIGHT

        var sprite_size = $Sprite2D.texture.get_size()
        var spawn_offset = sprite_size.x * 0.5

        bullet.activate(
                global_position + direction * spawn_offset,
                direction
        )

        shoot_timer.start()

func take_damage(amount: int) -> void:
        player_life -= amount
        print("[Player] player life: ", player_life)

        if player_life <= 0:
                _die()

func _die() -> void:
        print("game over!")
        get_tree().paused = true
```

bullet.gd

```
extends Area2D

@export var speed := 600.0
@export var lifetime := 1.5
@export var damage := 10

var direction := Vector2.ZERO
var active := false
var life_timer := 0.0
```

```
func activate(pos: Vector2, dir: Vector2) -> void:
	global_position = pos
	direction = dir.normalized()
	life_timer = 0.0
	active = true
	visible = true
	set_deferred("monitoring", true)
	set_physics_process(true)

func _physics_process(delta: float) -> void:
	if not active:
		return

	position += direction * speed * delta
	life_timer += delta

	if life_timer >= lifetime:
		deactivate()

func _on_body_entered(body: Node) -> void:
	if not active:
		return

	if body.is_in_group("enemy") and body.has_method("take_damage"):
		body.take_damage(damage)

	deactivate()

func deactivate() -> void:
	active = false
	visible = false
	set_deferred("monitoring", false)
	set_physics_process(false)
```

pool_manager.gd

```
extends Node

@export var bullet_scene: PackedScene
@export var pool_size := 30

var pool: Array = []
var container: Node

func setup(_container) -> void:
	container = _container
	create_pool()

func create_pool() -> void:
	for i in range(pool_size):
		var bullet = bullet_scene.instantiate()
		bullet.visible = false
		bullet.set_physics_process(false)
		container.add_child(bullet)
```

```
                pool.append(bullet)

func get_bullet():
        for bullet in pool:
                if not bullet.active:
                        return bullet

        return null
```

enemy.gd

```
extends CharacterBody2D

@export var max_health := 30
@export var speed := 80
@export var damage := 10
@export var attack_range := 140.0

var can_attack := true
var health: int
var player

enum State {
        CHASE,
        ATTACK
}

var state: State = State.CHASE

@onready var attack_timer := $AttackTimer

func _ready() -> void:
        add_to_group("enemy")
        health = max_health
        player = get_tree().get_first_node_in_group("player")

func _physics_process(_delta: float) -> void:
        if player == null:
                return

        var distance = global_position.distance_to(player.global_position)

        match state:
                State.CHASE:
                        update_chase(distance)
                State.ATTACK:
                        update_attack(distance)

        move_and_slide()

func update_chase(distance: float) -> void:
        var direction = (player.global_position - global_position).normalized()
        velocity = direction * speed

        if distance < attack_range:
```

```
                state = State.ATTACK

func update_attack(distance: float) -> void:
        velocity = Vector2.ZERO

        if can_attack:
                _attack()

        if distance > attack_range:
                state = State.CHASE

func _attack() -> void:
        if player.has_method("take_damage"):
                player.take_damage(damage)
                can_attack = false
                attack_timer.start()

func _on_attack_timer_timeout() -> void:
        can_attack = true

func take_damage(amount: int) -> void:
        health -= amount
        print("[Enemy] enemy HP: ", health)

        if health <= 0:
                _die()

func _die() -> void:
        var gm = get_tree().get_first_node_in_group("game_manager")
        if gm:
                gm.add_score(10)

        print("[Enemy] enemy died")
        queue_free()
```

enemy_spawner.gd

```
extends Node2D

@export var enemy_scene: PackedScene

func _on_timer_timeout() -> void:
        var enemy = enemy_scene.instantiate()

        enemy.global_position = Vector2(
                randf_range(100, 600),
                randf_range(100, 500)
        )

        get_tree().current_scene.add_child(enemy)
```

game_manager.gd

```
extends Node

signal score_changed(new_score)

var score := 0

func _ready() -> void:
	add_to_group("game_manager")

func add_score(amount: int) -> void:
	score += amount
	print("score: ", score)
	score_changed.emit(score)
```

ui.gd

```
extends Control

@onready var health_label = $HealthLabel
@onready var score_label = $ScoreLabel
@onready var game_over_label = $GameOverLabel

var player
var gm

func _ready() -> void:
	player = get_tree().get_first_node_in_group("player")
	gm = get_tree().get_first_node_in_group("game_manager")
	_update_score(0)

	if gm:
		gm.score_changed.connect(_update_score)

	game_over_label.visible = false

func _process(_delta: float) -> void:
	if player:
		health_label.text = "Health: " + str(player.player_life)

	if player and player.player_life <= 0:
		game_over_label.visible = true

	if Input.is_action_just_pressed("restart"):
		get_tree().paused = false
		get_tree().reload_current_scene()

func _update_score(new_score: int) -> void:
	score_label.text = "Score: " + str(new_score)
```

১১. Short Reflection Sheet

এই page-টা future revision-এর জন্য। চাইলে print করে বা notebook-এ নকল করে উত্তর লিখতে পারো।

প্রশ্ন	তোমার উত্তর
এই game-এ সবচেয়ে বড় breakthrough কী ছিল?	
কোন phase-এ সবচেয়ে বেশি confusion হয়েছিল?	
Pooling-এর আসল ধারণা এক লাইনে কী?	
Enemy FSM-এ state কেন দরকার?	
GameManager কে কেন দরকার ছিল?	
পরের game-এ কী discipline carry করবে?	

শেষ কথা: Game 8-এর সবচেয়ে বড় অর্জন feature না; বরং তুমি শিখেছ কীভাবে বড় system-কে ছোট, stable chunk-এ ভাগ করতে হয়। এই skill থাকলে পরের game-গুলো অনেক বেশি manageable হবে।